InfoMed: guia de entrada do parceiro de desenvolvimento

Para quem vai continuar o código de onde o outro parou. Siga na ordem; cada etapa tem um jeito de conferir se deu certo.

1. Visão geral em um minuto

- **O que é:** InfoMed, aplicativo de saúde pública (medicamentos nos postos, localização das unidades, agendamento de consultas, lembretes). É o TCC da dupla. Cidade-alvo: **Santa Bárbara d'Oeste (SP)**.
- **Stack:** React 18 + Vite + TypeScript + Tailwind (front), **Supabase** (PostgreSQL, login, segurança por usuário), Leaflet/OpenStreetMap (mapas, ainda não feito).
- **Origem do código:** o protótipo foi gerado no Figma Make e depois refatorado (limpeza de dependências, TypeScript, camadas).
- **Já funciona:** projeto versionado no GitHub, padrão Repository, banco com 9 tabelas e RLS, tela de Medicamentos lendo do banco, login e cadastro com aceite da LGPD, testes automatizados e a pasta `docs/` com as decisões.
- **Prazo:** cerca de 1 mês. Princípio: entregar menos, mas bem estruturado e documentado.
- **Divisão:** a dupla alterna o trabalho no código; cada um usa o seu próprio chat com o Claude. Por isso existe um arquivo de contexto (seção 8) e um diário de progresso (`docs/PROGRESSO.md`).

2. O que o dono do repositório precisa fazer antes

Quem criou o projeto faz estas 3 coisas **uma vez só**:

1. **Convidar o parceiro no GitHub:** no repositório, **Settings → Collaborators → Add people**, digitar o usuário ou e-mail do parceiro. O parceiro aceita o convite que chega por e-mail (sem aceitar, o repositório privado não abre).
2. **Convidar no Supabase (recomendado):** no painel do projeto, **Organization → Team → Invite**, com o e-mail do parceiro. Assim ele enxerga as tabelas e roda scripts SQL.
3. **Enviar as chaves do `.env` por canal privado** (mensagem direta, nunca em grupo público nem no GitHub): a `VITE_SUPABASE_URL` e a `VITE_SUPABASE_PUBLISHABLE_KEY`. A chave **secret** ou **service_role** não se envia para ninguém e não é usada no projeto.

3. Preparar a máquina (Windows)

Item	Onde baixar	Como conferir
Node.js (versão **LTS**)	nodejs.org	`node -v` mostra um número
Git	git-scm.com	`git --version` mostra um número
VS Code	code.visualstudio.com	abre normalmente
Conta no GitHub	github.com	consegue logar

Depois de instalar tudo, **reinicie o computador**.

Liberar scripts no PowerShell (erro comum no Windows). Abra o PowerShell como administrador (botão direito → Executar como administrador) e rode:

```
Set-ExecutionPolicy RemoteSigned
```

Responda `S` (ou `Y`). Depois feche essa janela.

Configurar o Git (uma vez):

```
git config --global user.name "Seu Nome"
git config --global user.email "seuemail@exemplo.com"
```

4. Baixar o projeto e rodar

Atenção: coloque o projeto **fora do OneDrive**. A pasta `node_modules` tem milhares de arquivos e o OneDrive trava tudo. Use `C:\projetos`.

1. Crie a pasta `C:\projetos`.
2. Abra o **VS Code** → menu **Terminal → New Terminal**.
3. Rode (troque pelo endereço real do repositório, que termina em `.git`):

```
cd C:\projetos
git clone https://github.com/USUARIO/NOME-DO-REPOSITORIO.git InfoMed
cd InfoMed
```

Se abrir uma janela do navegador pedindo login do GitHub, faça o login e autorize.

4. No VS Code: **File → Open Folder** → `C:\projetos\InfoMed`. Clique em **Yes, I trust the authors**.
5. Abra um terminal novo (**Terminal → New Terminal**). A linha deve começar com `PS C:\projetos\InfoMed>`. Rode:

```
npm install
```

6. Crie o arquivo de configuração: clique com o botão direito no `.env.example` → Copy → cole no mesmo lugar e **renomeie a cópia para `.env`**. Abra o `.env` e cole os dois valores recebidos do parceiro:

```
VITE_SUPABASE_URL=https://SEU-PROJETO.supabase.co
VITE_SUPABASE_PUBLISHABLE_KEY=sb_publishable_xxxxxxxxxxx
```

7. Rode o app:

```
npm run dev
```

Clique no link `http://localhost:5173/` segurando **Ctrl**.

Checklist: está tudo certo se...

- `npm run dev` abre o InfoMed no navegador;
- a tela de Medicamentos mostra a lista (vinda do banco);
- `npx tsc --noEmit` termina sem erros;
- `npm test` mostra os testes passando;
- `git status` **não** lista o arquivo `.env` (ele é privado e já está no `.gitignore`).

Sem o `.env`, o app ainda abre, mas usa dados de mentira (modo Mock). Isso é proposital.

5. Como o código está organizado

```
src/app/
  components/screens/   telas (vieram do Figma Make)
  components/shared/    BottomNav, Header, SearchBar...
  domain/               entidades e tipos (Medication, AuthUser)
  repositories/         interfaces + implementações (mock/ e supabase/)
  features/auth/        estado de login (AuthContext) e testes
  lib/supabase.ts       conexão com o Supabase
db/                     scripts SQL numerados (01, 02, 03, 04...)
docs/                   ADRs, diagramas, fichas, segurança, evidências
```

As regras de arquitetura (não quebrar):

1. **Tela nunca fala com o banco.** A tela usa uma *interface* de repositório; quem escolhe Mock ou Supabase é só o `repositories/index.ts`.
2. **Tipos e entidades ficam em `domain/`.** Nada de `any` novo.

3. **Mudança no banco vira um arquivo SQL novo e numerado em `db/`** (por exemplo `db/05_postos_reais.sql`). Nunca editar o `01_schema.sql` depois que foi rodado.
4. **Toda tabela com dados pessoais precisa de RLS** (`user_id = auth.uid()`).
5. **Interface em português**, com o visual do Figma (cores e componentes já existentes).
6. **Nenhum segredo no código.** O `.env` nunca vai para o GitHub.

6. Rotina de trabalho a dois

```
flowchart LR
  A[git pull] --> B[ler docs/PROGRESSO.md]
  B --> C[trabalhar com o Claude em uma tarefa]
  C --> D[tsc + npm test + build]
  D --> E[atualizar PROGRESSO.md e docs]
  E --> F[git add / commit / push]
  F --> G[avisar o parceiro]
```

Ao começar uma sessão

```
git pull
```

Depois leia o final de `docs/PROGRESSO.md` (o que o outro fez e o que falta). Se o `package.json` mudou, rode `npm install` de novo.

Durante o trabalho

- Uma tarefa por vez, em passos pequenos.
- Para testar: `npx tsc --noEmit`, `npm test` e `npm run build`. Só faça commit se os três passarem.
- Se precisar mexer no banco, crie o SQL novo em `db/` e **avise o parceiro**: o Supabase é um só para a dupla, então um roda o script e o outro apenas faz `git pull`.

Ao terminar a sessão

1. Peça ao Claude: *"resuma o que fizemos para eu colar no docs/PROGRESSO.md, no formato do arquivo"* e cole no final do arquivo.
2. Se foi uma decisão técnica nova, crie um ADR; se foi uma funcionalidade concluída, preencha uma ficha em `docs/features/`.
3. Envie:

```
git add .
git commit -m "feat: descrição curta do que foi feito"
git push
```

4. Mande uma mensagem ao parceiro dizendo o que mudou (principalmente se há SQL novo para rodar).

Branches (recomendado se os dois mexerem ao mesmo tempo)

```
git checkout -b feat/postos-mapa
# ...trabalha, commita...
git push -u origin feat/postos-mapa
```

Depois abra um **Pull Request** no GitHub, o outro confere e faz o *Merge*. Para voltar ao principal: `git checkout main` e `git pull`.

Arquivos que mais geram conflito: `App.tsx`, `repositories/index.ts` e `package.json`. Combinem antes de editar esses.

Se aparecer conflito: o VS Code marca o trecho com `<<<<<<<` e `>>>>>>>` e mostra botões *Accept Current / Accept Incoming / Accept Both*. Escolha, salve, depois rode `git add .` e `git commit`. Na dúvida, pergunte ao parceiro antes de apagar o trabalho dele.

7. Como usar o Claude sem gerar confusão

O Claude de cada um **não lembra** das conversas do outro. Todo o contexto precisa estar escrito. Por isso:

1. Abra um **chat novo** no Claude.
2. **Cole o prompt de contexto da seção 8** e, se possível, anexe o arquivo `CONTEXTO-IA.md` da raiz do repositório (ele é a versão completa e sempre atualizada).
3. Diga em seguida qual é a tarefa (veja a lista na seção 9).
4. Quando pedir código, peça também **onde criar o arquivo** e **como testar**.
5. Se o Claude sugerir algo que quebre as regras da seção 5 (trocar de tecnologia, ligar tela direto no banco, usar a chave secret, editar o `01_schema.sql`), responda: *"isso viola as regras do CONTEXTO-IA.md, refaça seguindo o padrão Repository"*.
6. **Nunca cole chaves, senhas ou o conteúdo do `.env` no chat.**
7. Se o Claude disser que algo "já existe" e você não achar no projeto, rode `git log --oneline` e abra o arquivo: confie no repositório, não na memória da IA.
8. Ao final da sessão, peça o resumo para o `PROGRESSO.md`.

8. Prompt de contexto (copiar e colar no início de cada chat novo)

```
Você é meu mentor e par de programação no TCC "InfoMed", um app de saúde pública (medicamentos nos postos, localização de unidades, agendamento de consultas e lembretes) para Santa Bárbara d'Oeste (SP). Eu e um colega alternamos o trabalho no mesmo repositório; este chat é só meu, então todo o contexto vem abaixo e do arquivo CONTEXTO-IA.md (se eu anexar).

STACK (já decidido, não troque): React 18 + Vite + TypeScript + Tailwind; Supabase (PostgreSQL, Auth, RLS); Leaflet + OpenStreetMap (a fazer); Vitest para testes; GitHub; PWA.

ESTADO ATUAL: protótipo do Figma Make refatorado e versionado; padrão Repository aplicado (interface + Mock + Supabase); banco com 9 tabelas e RLS; tela de Medicamentos lendo do banco; login/cadastro com aceite LGPD; documentação em docs/ (ADRs 001-008). Leia docs/PROGRESSO.md para ver o que o colega fez por último.

REGRAS QUE VOCÊ DEVE RESPEITAR:
1. Telas nunca acessam o banco; usam interfaces em src/app/repositories (implementações em mock/ e supabase/, escolhidas em repositories/index.ts).
2. Entidades e tipos em src/app/domain; sem 'any' novo.
3. Mudanças no banco = novo arquivo SQL numerado em db/ (05, 06...); nunca editar o 01_schema.sql.
4. Toda tabela de dados pessoais com RLS (user_id = auth.uid()).
5. Nunca usar nem pedir a chave service_role/secret; nunca versionar o .env.
6. Interface em português, mantendo o visual do Figma.
7. Não adicione bibliotecas sem justificar; não reintroduza MUI, emotion e similares.
8. Antes de dizer que terminou, confira: npx tsc --noEmit, npm test e npm run build.
9. Cada decisão técnica nova vira um ADR em docs/decisions; cada funcionalidade concluída vira uma ficha em docs/features.
10. Se algo no projeto contradizer o que você lembra, pergunte-me ou peça para eu mostrar o arquivo; não invente arquivos.

COMO RESPONDER: passo a passo, curto, dizendo exatamente qual arquivo abrir/criar e o que colar; um passo de cada vez; assuma Windows + VS Code + terminal do VS Code. Ao final de cada etapa, me dê um resumo para colar em docs/PROGRESSO.md.

TAREFA DE HOJE: <descreva aqui>
```

9. Próximas tarefas, em ordem

Pegue a próxima da fila, avise o parceiro no grupo e registre em `PROGRESSO.md`.

#	Tarefa	Pronto quando...	Disciplina
T1	Confirmar login e cadastro no ambiente real; rodar `db/04_auth.sql` no Supabase se ainda não rodou; coletar prints	Cadastro grava `profiles.lgpd_consent_at`; prints em `docs/evidence/`	Segurança
T2	Teste de isolamento com 2 contas usando `db/03_teste_rls.sql`	Tabela de resultados preenchida em `docs/security/testes-rls.md`	Segurança
T3	**Postos reais de Santa Bárbara d'Oeste:** levantar UBS e unidades (nome, endereço, latitude/longitude, horário, serviços) em fontes públicas (CNES, OpenStreetMap, prefeitura), conferir manualmente e criar `db/05_postos_reais.sql` que substitui as unidades fictícias	Tabela `health_units` com dados reais conferidos; fontes citadas no doc	Nuvem / BI
T4	`HealthUnitRepository` (interface, Mock, Supabase) e telas de Postos lendo do banco	Lista e detalhe do posto vêm do banco	Padrões de Projeto
T5	Mapa com Leaflet na lista e no detalhe do posto (ADR-005)	Marcadores no mapa de Santa Bárbara d'Oeste	Multi Plataforma
T6	Migrar a navegação para rotas (react-router) e remover a moldura de iPhone no celular	Botão voltar funciona; app ocupa a tela no celular	Multi Plataforma
T7	Agendamento ponta a ponta (especialidade → médico → data/hora → confirmação) com padrão **Factory** e a função `booked_slots`	Consulta salva no banco e visível só para o dono	Padrões de Projeto
T8	Lembretes com padrão **Observer** e notificações	Lembrete persiste e gera notificação	Padrões de Projeto

#	Tarefa	Pronto quando...	Disciplina
T9	Filtros dos postos com padrão **Strategy**	Filtrar por distância, estoque e horário	Padrões de Projeto
T10	CI no GitHub Actions: tipos, testes e build; depois deploy (ADR-006)	Pipeline verde, com print	Entrega Contínua
T11	Funcionalidade de IA (sintoma → especialidade) em Edge Function, com aviso de que não substitui consulta	Chave da IA só no servidor	IA
T12	Dashboard de BI com views SQL	Painel com dados de postos, medicamentos e consultas	BI e Big Data
T13	Pesquisa de usabilidade (SUS) com 15 a 30 pessoas	Respostas coletadas e análise descritiva	Estatística

Aviso importante: o estoque de medicamentos por unidade é **simulado** (não existe fonte pública confiável por posto). Isso deve constar no TCC. Os dados das unidades (T3) são reais e precisam ser conferidos antes da apresentação.

10. Documentar enquanto desenvolve

Feature sem documentação não está concluída.

- **Decisão técnica nova** → `docs/decisions/ADR-0XX-nome.md` (use os existentes como modelo: contexto, decisão, alternativas, consequências, evidências).
- **Funcionalidade concluída** → `docs/features/F0X-nome.md` (modelo em `features/_modelo.md`).
- **Prints** → `docs/evidence/` (tela funcionando, pipeline verde, políticas do banco...).
- **Diagramas** → `docs/diagrams/` em Mermaid (o GitHub desenha automaticamente).
- **Diário** → `docs/PROGRESSO.md`, uma entrada por sessão.

11. Problemas comuns e soluções

Sintoma	Causa provável	Solução
`npm` bloqueado, "execução de scripts desabilitada"	Política do PowerShell	`Set-ExecutionPolicy RemoteSigned` como administrador

Sintoma	Causa provável	Solução
`package.json` não encontrado	Terminal aberto na pasta errada	Confira se a linha termina em `\InfoMed>`; use `cd C:\projetos\InfoMed`
Tudo lento, erros estranhos	Projeto dentro do OneDrive	Mover para `C:\projetos`
App abre com dados de mentira	Falta o `.env` ou está vazio	Criar o `.env` (seção 4, passo 6) e reiniciar o `npm run dev`
Mudei o `.env` e nada mudou	O Vite só lê o `.env` ao iniciar	Parar (Ctrl+C) e rodar `npm run dev` de novo
`git push` recusado	O parceiro enviou antes	`git pull`, resolva conflitos se houver, e `git push`
Repositório "não encontrado" ao clonar	Convite do GitHub não aceito	Aceitar o convite no e-mail
Erro de permissão ao rodar consulta no app	Falta política ou GRANT na tabela nova	Criar SQL novo com `grant` e `policy` (ver `01_schema.sql` como exemplo)
Erro ao instalar o Vitest	Versão nova exige Vite mais recente	Usar `vitest@3` (já está no `package.json`)
Digitei a chave secret por engano no código	—	Avisar o parceiro e **gerar nova chave** no painel do Supabase antes de continuar

12. Referências rápidas

Item	Onde
Repositório	(colar o link aqui)
Projeto Supabase	(colar o link do painel aqui)
Protótipo no Figma Make	(colar o link aqui)
Contexto para IA	`CONTEXTO-IA.md` (raiz do repositório)
Diário de progresso	`docs/PROGRESSO.md`
Relatório geral do que já foi feito	Documento "InfoMed: relatório de progresso do desenvolvimento"